UN MANUALE DI BM

bm: il bootloader e il sistema operativo

Cos'è bm, come si avvia, cosa fanno il suo menu e i suoi tasti, e quali file tiene sulla scheda SD.

Indice

Cos'è bm 4

Cosa fa 4

Da sapere 4

La catena di avvio 5

Sul Pi Zero W 5

Caricare un kernel dal PC 5

Sull'RGB30 6

Cosa fa il kernel all'avvio 7

Il menu e le sue schede 8

I tasti del menu 8

Da sapere 8

Impostazioni 8

I tasti di sistema 10

Da sapere 10

Scoprire i tasti di un gioco 11

Il monitor 12

Come aprirlo 12

Comandi 12

Una riga di comandi 13

I file sulla scheda SD 14

Impostazioni e config.txt 15

Cambiare un'impostazione dal PC 15

Schermate 17

Da sapere 17

Provalo, passo per passo 17

Salvataggi 18

Crash e log **19**

Un crash 19

Dove sono i log 19

Aggiornare **20**

Dalla console 20

Dal PC 20

Punti aperti **21**

LEGGIMI

Cos'è. Un giro guidato dentro bm stesso: l'avvio, il kernel, il menu, i tasti di sistema, il monitor e i file sulla scheda. È scritto per un utente o uno sviluppatore curioso.

Come usarlo. Leggi il capitolo 1 per l'idea, poi salta a ciò che ti serve. Ogni procedura è un elenco di passi numerati; tutto ciò che sta in un riquadro con bordo e ombra va copiato così com'è.

Cosa manca. Ogni fatto viene dal repository. Dove qualcosa non era scritto o non è stato provato sull'hardware, il manuale mostra un'etichetta evidenziata DA COMPLETARE; nulla è inventato.

Cos'è bm

bm (BareMetal) è una console per giochi e un ambiente di sviluppo che gira sul **Raspberry Pi Zero W** e sul **PowKiddy RGB30**. Sotto non c'è nessun sistema operativo: il repository lo descrive come un kernel bare metal, senza Linux, senza demoni in background e senza lo scheduler di un sistema operativo. Il kernel parla da solo con i registri dell'hardware.

Scheda	Chip	Immagine sulla scheda SD	Build
Pi Zero W (anche Pi 1)	BCM2835, ARM1176	`kernel.img`	`make`
PowKiddy RGB30	RK3566, Cortex-A55 (AArch64)	`kernel8.img`	`make TARGET=rgb30`

Cosa fa

Il kernel parte, mostra un menu e lancia le **cartucce**: singoli file (`.bm` , e `.b16` sull'RGB30) che contengono il codice Lua 5.4, gli sprite, le mappe, le mesh 3D, le animazioni e i suoni di un gioco. Lo stesso menu apre anche gli editor della suite (la scheda **Dev**), il Market e le Impostazioni.

Da sapere

- Un solo menu serve entrambe le schede (`src/kernel/menu_ui.c`): sul Pi è 640x360, sull'RGB30 è 360x360 mostrato al doppio della grandezza.
- Il repository dice che il codice del Pi Zero 2 W resta nell'albero ma è spento nella build normale.
- Un `.b16` non sostituisce mai un `.bm` : sono due edizioni separate di un gioco.

La catena di avvio

Sul Pi Zero W

1. Accendi. Il firmware del Pi legge la scheda FAT32. Servono `bootcode.bin`, `start.elf` e `fixup.dat` nella radice della scheda.
2. Il firmware legge `config.txt` nella radice (il file del firmware, non quello di bm: vedi il capitolo 9) e carica il file indicato da `kernel=`, cioè `kernel.img`, all'indirizzo 0x8000.
3. Il codice di bm prende il controllo: gira solo il core 0, gli altri core sono parcheggiati, si preparano gli stack e si azzera la memoria di lavoro.
4. Parte il kernel (capitolo successivo) e si arriva al menu.

Il `config.txt` fornito con bm chiede al firmware queste cose:

Riga	Cosa fa
`kernel=kernel.img`	le schede BCM2835 (Pi Zero, Zero W, Pi 1) avviano questo file
`hdmi_force_hotplug=1`	manda sempre l'HDMI, anche se non rileva un monitor
`hdmi_drive=2`	modo HDMI (non DVI), così la TV riceve l'audio
`disable_overscan=1`	usa tutta l'area dello schermo
`scaling_kernel=8`	la GPU ingrandisce l'immagine 640x360 con pixel netti
`hdmi_group=1` e `hdmi_mode=16`	sempre 1080p60, così un'immagine 16:9 mantiene la forma
`disable_splash=1`	salta lo splash arcobaleno
`enable_uart=1`	tiene stabile il clock del core per la console seriale

Caricare un kernel dal PC

Il repository costruisce anche un piccolo **chainloader** (`chainloader/`, costruito da `make` come `build/chainloader.img`). Messo sulla scheda al posto del kernel, aspetta sulla porta seriale, riceve un `kernel.img` dal PC, ne controlla il CRC e salta a 0x8000. Mentre aspetta fa lampeggiare il LED e manda tre byte ogni secondo.

> **NOTA**
> Il comando `r` del monitor riavvia il Pi, e il chainloader chiede allora un nuovo kernel. Così uno sviluppatore prova una build senza toccare la scheda.

DA COPIARE

```
make sdcard-chainloader
make run-serial
```

`make sdcard-chainloader` prepara il contenuto della scheda con il chainloader al posto del kernel. `make run-serial` invia `build/kernel.img` con `tools/bm_load.py`, poi apre un terminale. La

linea seriale è a 115200 baud, 8N1, su GPIO14 e GPIO15.

Sull'RGB30

L'RGB30 non usa il firmware del Pi. L'ordine è: la boot ROM del chip, poi `idbloader.img` (inizializzazione della memoria), poi `u-boot.itb` (U-Boot), poi `/extlinux/extlinux.conf` sulla prima partizione della scheda nello slot TF1, che avvia `kernel8.img`. I file del bootloader vengono dall'immagine ufficiale ROCKNIX e stanno prima della partizione, quindi non devi formattare né ripartizionare la scheda. Accendi con il **tasto di accensione**.

Cosa fa il kernel all'avvio

Sul Pi, `main.c` fa queste cose, in quest'ordine:

1. Avvia il LED e la porta seriale, poi lo schermo e lo splash (il logo); le righe dell'avvio vanno alla console.
2. Prepara la memoria: lo heap, il clock dell'ARM al massimo offerto dal firmware e la MMU.
3. Riconosce la scheda e la stampa con il suo SoC e il codice di revisione.
4. Prepara il LED della scheda, poi gli interrupt e il timer.
5. Prova ad armare il watchdog (3 secondi). Il log di avvio dice `watchdog: on, 3000 ms` quando è armato. Se il controllo fallisce, dice "not available, no freeze guard" (in un emulatore: "resets at once on RSTC (emulator), no freeze guard"). Il Pi Zero W è armato; questo manuale non lo ha verificato su una console vera.
6. Avvia l'audio, il DMA e l'USB.
7. Carica l'elenco delle cartucce e `bm/config.txt` .
8. Trasforma il resoconto di crash dell'avvio precedente in un report (capitolo 12).
9. Avvia il Bluetooth se c'è un pad abbinato, poi WiFi ed Ethernet.
10. Apre il menu delle cartucce. Il monitor è la console di testo che gira dopo.

Il README dice che il Pi arriva al menu in circa due secondi. **[DA COMPLETARE: il tempo di avvio dell'RGB30 non è stato misurato nel repository]**

Sull'RGB30 un LED ti dice come va l'avvio quando non hai un cavo seriale:

LED	Significato
rosso fisso	avvio in corso; se resta così, il kernel si è bloccato presto
verde fisso	bm sta girando e va tutto bene
verde lampeggiante lento	altro: avvio non finito, SD mancante o illeggibile, batteria scarica, un kernel in arrivo, un aggiornamento in installazione
rosso fisso e verde lampeggiante	lo schermo ha avuto problemi: leggi `bm/bootlog.txt`
rosso lampeggiante N volte, pausa	eccezione fatale N (1 sincrona, 4 SError, 9 panic)

Il menu e le sue schede

Il menu è la prima cosa che vedi. Si apre su **Games**.

Scheda	Cosa contiene
Market	il catalogo di giochi e app da scaricare; due pagine, **Games** e **Apps**
Games	le cartucce sulla scheda
Dev	gli strumenti: SDK, editor, Sound, Studio, Animator, Mesh, Pixel
Settings	l'ultima scheda: il suo pannello si apre al posto di un elenco

Il Market si carica solo mentre la sua scheda è attiva e non blocca mai il menu.

[DA COMPLETARE: shots/boot-os-menu.png]

I tasti del menu

Tasto	Cosa fa
frecce o `w` `a` `s` `d`	scegli
`Enter`	gioca, apri, sì
`c`	opzioni del gioco
`q` / `e`	la scheda prima, dopo
`Tab`	la scheda successiva
`1` - `5`	Market ... Settings
`r`	rileggi la scheda SD
mouse	il clic gioca; il clic destro torna indietro

Le opzioni del gioco (`c`) mostrano i byte dei salvataggi e una riga *Delete the save data*. Sull'RGB30 le stesse opzioni si aprono con **X** e hanno anche *Send to a nearby console*; provato solo in QEMU (non verificato su hardware).

Da sapere

- Sull'RGB30, **B** conferma e **A** torna indietro. Imposta `confirm=a` in `bm/config.txt` per scambiarli, oppure usa Settings > Controllers > *Confirm button*.
- Sull'RGB30 la barra in alto mostra solo WiFi e batteria. Games elenca i file `.b16` e mostra i `.bm` per le prove: `show_bm=0` li nasconde. Il Market elenca i file `.b16` e `.bm` . Un `.bm` non fatto per l'RGB30 è grigio e ne dice il motivo. `market_bm=0` nasconde i `.bm` .
- Gli editor lavorano solo sui progetti. Un `.bm` è in sola lettura; Ctrl+B in un editor costruisce il `.bm` definitivo.

Impostazioni

Le Impostazioni hanno queste pagine: Controllers, WiFi and network (Network su una scheda senza radio), Screen, Sound, Updates, Reports e System, poi le righe *Restart* e *Shut down*. System mostra la versione, la scheda, il tempo di accensione, la memoria in uso, la CPU, la scheda SD e la batteria, e ha *Log since boot* e *Open the monitor*.

System ha anche *SD map*: una barra di ciò che c'è sulla scheda, divisa in Kernel, bm (giochi), bme (progetti), glb e bmm (modelli), Other e Free, la dimensione di ciascuno, i file e i kilobyte di ogni cartella, e un elenco *Other files* con i file che il menu non sa mostrare, i più grandi per primi. Il menu legge la scheda quando è libero e mai mentre gira un gioco o uno strumento; la riga *Scan* mostra lo stato e la percentuale. La console tiene l'ultimo risultato in `bm/sdmap.txt`. La SD map non è verificata su hardware (Pi Zero W o RGB30).

[DA COMPLETARE: shots/boot-os-settings.png]

I tasti di sistema

I tasti di sistema funzionano nel menu, dentro i giochi e dentro gli strumenti. La maggior parte è definita in un'unica tabella, `src/kernel/syskeys.c`, che elenca i tasti mostrati sullo schermo mentre tieni premuto F12. Print Screen non è in quella tabella: giochi e app lo gestiscono a parte (`src/bm/runtime.c`).

Tasto	Cosa fa
`F12` (tieni premuto)	mostra i tasti
`Esc`	il menu, o indietro; dentro un gioco è Start (il menu del gioco)
`Ctrl+Esc`	torna al menu di bm; il gioco resta sospeso
`Ctrl+Shift+Esc`	il monitor
`F11`	overlay delle prestazioni: spento, in alto, in basso
`F1 - F4`	le pagine (strumenti)
`F5` / `Ctrl+R`	prova il gioco (strumenti)
`F6`	l'assistente (strumenti)
`Ctrl+S`, `Ctrl+Shift+S`, `Ctrl+O`, `Ctrl+N`	salva, salva con nome, apri, nuovo (strumenti)
`Ctrl+Z`, `Ctrl+Y`, `Ctrl+X`, `Ctrl+C`, `Ctrl+V`, `Ctrl+F`	annulla, ripeti, taglia, copia, incolla, cerca (strumenti)
`Print Screen`	salva una schermata (capitolo 10)

Su un pad, **PS** o **Start+Select** fanno ciò che fa Ctrl+Esc. Il sistema segna F12, Ctrl+Esc, Ctrl+Shift+Esc e F11 come tasti suoi. Un gioco sospeso si può riprendere dalla sua copertina.

Da sapere

- In una partita di rete il sistema non sospende il gioco: chiede "Leave the match?" e, se rispondi sì, il gioco si chiude.
- L'overlay mostra i fotogrammi al secondo, i millisecondi di lavoro in un fotogramma e il carico di CPU e GPU. Dura una sola esecuzione; ogni gioco parte come dice la chiave `perf` di `bm/config.txt` (spento di default; la RGB30 non ha F11, quindi lì imposta la chiave).
- Dalla linea seriale, Ctrl+\ funziona come Ctrl+Esc.

Scoprire i tasti di un gioco

Ogni gioco ha i suoi tasti, e F12 li elenca accanto a quelli del sistema.

1. Avvia la console con una tastiera USB o Bluetooth collegata. Dovresti vedere il menu sulla scheda Games.
2. Scegli un gioco con le frecce. Il nome del gioco compare su una pastiglia sotto le schede.
3. Premi `Enter`. Il gioco parte.
4. Tieni premuto `F12`. Compare un elenco a tutto schermo intitolato `Keys` con `(F12 held)`, con prima i tasti di sistema.
5. Leggi nell'elenco i tasti propri del gioco.
6. Rilascia `F12`. L'elenco sparisce e il gioco continua.
7. Premi `Ctrl+Esc` per uscire. Torni al menu; il gioco è sospeso e la sua copertina ora dice Resume.

[DA COMPLETARE: shots/boot-os-keys.png]

PROVA

Fai lo stesso nel menu stesso: tieni premuto F12 sulla scheda Games. Ottieni due gruppi, `bm` (i tasti di sistema) e `Menu` (i tasti del menu).

NOTA

Se l'elenco non mostra i tasti di un gioco, il gioco non ne ha dichiarati. **[DA COMPLETARE: come vedere i tasti di un gioco quando è collegato solo un pad; F12 è un tasto di tastiera]**

Il monitor

Il monitor è la console di testo di bm. Ha un prompt `>`, prende un tasto per comando e scrive sullo schermo, sulla porta seriale e sulla console di rete.

Come aprirlo

- Premi `Ctrl+Shift+Esc` ovunque.
- Oppure scegli Settings > System > *Open the monitor*.
- Oppure collegati dal PC: `python3 tools/bm_net.py IP` mostra il monitor di una console in rete.

Digita `h` per l'aiuto. `M` torna al menu; `Esc` da solo non fa nulla al prompt.

Comandi

L'aiuto del monitor elenca molti tasti. Questi sono quelli che contano per chi comincia:

Tasto	Cosa fa
`h` o `?`	l'aiuto
`M`	il menu delle cartucce
`f` / `F`	elenca le cartucce / rileggi la scheda SD
`i`	informazioni di sistema
`m`	uso dell'heap
`c`	pulisci lo schermo
`l`	prompt Lua (`Esc`, `Ctrl-D` o `exit()` tornano qui)
`L`	layout della tastiera, italiano o US
`V`	disegno dei `.bm`, diretto sullo schermo o via RAM
`y` / `Y`	scansione USB / prova degli input
`T` `K` `O`	Bluetooth: abbina un pad, una tastiera, un mouse
`P`	Bluetooth: dimentica tutti i dispositivi abbinati (chiede prima)
`W`	WiFi: avvia, elenca le reti, entra in una
`u`	aggiorna da GitHub (chiede prima)
`o`	tutto ciò che è stato stampato dall'avvio
`Z`	il log dall'avvio, come report
`r`	riavvia
`X`	prove di crash: poi `u`, `a`, `b`, `s` o `f`

Altri tasti lanciano test e benchmark: `k`, `p`, `D`, `g`, `j`, `R`, `s`, `d`, `t`, `a`, `b`. Altri aprono gli editor: `e`, `A`, `3`, `4`, `5`, `6`, `C`, `I`.

Una riga di comandi

Digita `:` e poi dei comandi separati da `;`. Girano uno dopo l'altro, e la riga si ferma al primo che fallisce.

Comando	Cosa fa
`b3d tests=a,b profiles=P`	il 3D Bench, tutto o in parte
`play NOME`	avvia un gioco per nome
`set key=value ...`	cambia le chiavi di `bm/config.txt` fino al riavvio
`save`	scrive `bm/config.txt`
`render` , `room` , `log` , `send` , `reboot`	benchmark, il log come report, invia i report in attesa, riavvia

DA COPIARE

```
:set volume=5; save
```

[DA COMPLETARE: shots/boot-os-monitor.png]

ATTENZIONE

`X` e poi `f` blocca la console di proposito, per provare la schermata di crash. Usalo solo quando vuoi fare quella prova.

I file sulla scheda SD

La scheda è FAT32. I nomi seguono la regola 8.3 quando è la console a scriverli.

Percorso	Cos'è
`bootcode.bin`, `start.elf`, `fixup.dat`	il firmware del Pi
`config.txt` (radice)	la configurazione del firmware
`kernel.img`	bm per il Pi Zero W e il Pi 1
`kernel7.img`	il kernel del Pi Zero 2 W, solo se costruito con ZER02=1
`kernel8.img`	bm per l'RGB30
`carts/`	le tue cartucce (`carts/nano8/` per le cartucce PICO-8)
`bm/config.txt`	le impostazioni di bm
`bm/ca.pem`	i certificati per HTTPS
file di firmware in `bm/`	firmware dei chip WiFi e Bluetooth
`bm/save/`	i salvataggi dei giochi
`bm/shots/`	le schermate
`bm/reports/`	report di test e di crash in attesa di invio
`bm/backup/`	i vecchi kernel, tenuti da un aggiornamento
`bm/lib/`	file di risorse inviati dal PC
`bm/bootlog.txt`, `bm/bootprev.txt`, `bm/lastrun.txt`	i log di avvio dell'RGB30 (capitolo 12)

NOTA

Una cartella chiamata `bm33`, di prima che il progetto fosse rinominato, viene spostata in `bm/` da `make install`.

Impostazioni e config.txt

Non confondere i due `config.txt`:

- quello nella **radice** della scheda appartiene al firmware del Pi (capitolo 2);
- quello in ** `bm/` ** appartiene a bm. Ha righe `key=value`. Le chiavi sconosciute vengono conservate così come sono.

Le Impostazioni, `set` / `save` della riga del monitor e gli strumenti del PC scrivono `bm/config.txt`. Queste sono chiavi nominate nel repository:

Chiave	Cosa fa
`layout`	layout della tastiera, `it` o `us`
`draw`	disegno dei `.bm`: `ram` o `direct`
`perf`	overlay delle prestazioni: `off`, `top`, `bottom`
`volume`	il volume
`sound`	`8bit` o `hifi`
`confirm`	`b` o `a`: quale pulsante conferma
`wifi_ssid`, `wifi_psk`	la rete a cui collegarsi
`wifi_boot=0`	non collegarsi alla rete all'avvio
`show_bm=0`	nasconde i file `.bm` nella scheda Games dell'RGB30
`market_bm=0`	nasconde i file `.bm` nel Market dell'RGB30
`bm_screen`	`1280x720` o `640x360`: lo schermo dell'SDK e degli editor (default 1280x720 con la GPU)
`menu_scale=1`, `2`, `3`	il menu del Pi a 640x360, 1280x720 (default con la GPU) o 1920x1080
`battery_icon=0`	spegne l'icona di batteria scarica (RGB30)
`mouse=off`	spegne il puntatore

Le chiavi il cui nome finisce con `_token`, `_psk`, `_password` o `_key` sono segreti: non vengono mai mostrate né inviate fuori. Tieni chiavi e token solo in questo file o nell'ambiente, mai in altri file.

DA COPIARE

```
wifi_ssid=LaMiaRete
wifi_psk=la mia password
```

Cambiare un'impostazione dal PC

1. Assicurati che la console sia in rete e annota il suo indirizzo IP (Settings > WiFi and network lo mostra).
2. Sul PC lancia lo strumento. Una riga `key=value` imposta una chiave; `key=` la rimuove.
3. La console risponde con le impostazioni come sono ora, con i segreti nascosti.

DA COPIARE

```
./easy_install.sh config
python3 tools/bm_net.py IP --config
```

Schermate

Premi `Print Screen` sulla tastiera, oppure tieni premuto **Select** per 3 secondi su un pad (Share su un DS4). La console salva lo schermo del gioco, com'è stato disegnato, come PNG in `bm/shots/`. Gli overlay, l'icona della batteria, il puntatore e il riquadro del messaggio non sono nell'immagine.

Da sapere

- Il nome è formato dalle prime lettere del titolo del gioco e da un numero: `pong0001.png`. Il numero sale da solo; bm non sovrascrive mai un'immagine. Il Pi non ha un orologio, quindi non c'è la data.
- Un riquadro dice `Screenshot saved` e il nome del file per due secondi.
- Le schermate del menu stesso non si possono ancora salvare. Quelle di giochi e app sì.
- Il tasto Print Screen non ha ancora la sua immagine nell'elenco di F12.

Provalo, passo per passo

1. Avvia un gioco dal menu.
2. Premi `Print Screen`. Compare un riquadro con `Screenshot saved`.
3. Spegni la console e metti la scheda nel computer.
4. Apri `bm/shots/`. L'immagine è lì.

PROVA
Scatta tre immagini dello stesso gioco. Ottieni `0001`, `0002` e `0003` con lo stesso prefisso.

[DA COMPLETARE: verificare Select tenuto 3 s sul PowKiddy RGB30; non ancora provato sulla console]

Un gioco scatta da solo con `shot()`; vedi il manuale dell'API.

Salvataggi

Un gioco conserva i suoi dati con `save()` e `saved()`. Ogni gioco ha **8 slot**, ciascuno una tabella fino a 32 KB, in `bm/save/`.

- Lo slot 1 è `xxxxxxxx.sav`; gli slot da 2 a 8 sono da `.S02` a `.S08`.
- `XXXXXXXX` deriva dal titolo e dall'autore del gioco. Se cambi uno dei due, il gioco riparte da zero.
- Le opzioni di un gioco (`c` sulla copertina) mostrano i byte usati e *Delete the save data* cancella tutti gli slot.
- Gli aggiornamenti e `make install` non toccano mai impostazioni e salvataggi.

Per fare una copia dei salvataggi di un gioco, copia i suoi file da `bm/save/` sul PC.

Crash e log

Un crash

- Un'eccezione fatale mostra una schermata rossa con i registri e l'elenco degli indirizzi da cui è stata chiamata, anche durante un gioco.
- Gli ultimi 4 KB stampati e una traccia di breadcrumb restano in memoria attraverso un reset. Al boot successivo diventano un report in `bm/reports/`, inviato quando la rete è attiva. La rete e `report()` chiedono il tuo permesso la prima volta.
- Dopo 20 secondi di funzionamento, lo schermo dice "Restarting in 60 s: this screen goes out as a report", il LED lampeggia un codice e la console si riavvia. Vale anche sul Pi Zero W: il riavvio scrive direttamente nei registri di gestione dell'alimentazione (`src/kernel/exceptions.c`, `src/drivers/watchdog.c`), quindi non dipende dal controllo del watchdog fatto all'avvio.
- **[DA COMPLETARE: cosa significa ogni numero di lampeggi del LED sul Pi Zero W; sono scritti solo i codici dell'RGB30]**

[DA COMPLETARE: shots/boot-os-crash.png]

Un errore Lua non manda in crash la console: la cartuccia si ferma e l'errore, con la sua riga, compare sullo schermo. Viene fermato anche un gioco che supera 20 milioni di istruzioni Lua in un fotogramma.

Dove sono i log

Dove	Cosa contiene
Settings > System > *Log since boot*	tutto ciò che è stato stampato da quando la console è partita
monitor `o`	lo stesso, a scorrimento
monitor `Z`	lo stesso, inviato come report
`bm/bootlog.txt` (RGB30)	il log di questo avvio, riscritto a più passi
`bm/bootprev.txt` (RGB30)	il log dell'avvio precedente, come è finito e le sue ultime righe
`bm/lastrun.txt` (RGB30)	l'intero log di una sessione finita con un riavvio o uno spegnimento fatto da bm

[DA COMPLETARE: un file di log sulla scheda SD per il Pi Zero W]

Per leggere i file, spegni la console, metti la scheda nel PC e apri `bm/`. Leggili prima di riaccendere: ogni avvio riscrive `bootlog.txt`.

Aggiornare

Dalla console

1. Collega la console al WiFi (Settings > WiFi and network). Dovresti vedere un indirizzo nella riga *Address*.
2. Apri Settings > Updates e scegli *Check for updates*. Legge l'ultima release firmata e dice se è più recente e quali file cambiano.
3. Scegli *Install the update*. Scarica e controlla ogni file prima di scrivere qualsiasi cosa.
4. Aspetta. Il vecchio kernel va in `bm/backup/` , il kernel della scheda viene scritto per ultimo e la console si riavvia.

Se qualcosa fallisce, non è stato scritto nulla finché i file non sono stati controllati. Il tasto `u` del monitor fa lo stesso, chiedendo prima.

Dal PC

`./easy_install.sh` mostra un menu: aggiornare il kernel in rete o sulla scheda, un'installazione completa, un'immagine disco, inviare un file, il monitor, la config. Lo stesso strumento può rimettere una release più vecchia (`rollback`).

DA COPIARE

```
./easy_install.sh kernel
./easy_install.sh install
python3 tools/bm_net.py IP --kernel build/kernel.img
```

ATTENZIONE

Sull'RGB30, se lo schermo resta nero dopo un riavvio, leggi `bm/bootlog.txt` sul PC prima di riaccendere.

Punti aperti

- **Cosa fa il kernel all'avvio**: il tempo di avvio dell'RGB30 non è stato misurato nel repository
- **Il menu e le sue schede**: immagine da fare: shots/boot-os-menu.png
- **Il menu e le sue schede**: immagine da fare: shots/boot-os-settings.png
- **Scoprire i tasti di un gioco**: immagine da fare: shots/boot-os-keys.png
- **Scoprire i tasti di un gioco**: come vedere i tasti di un gioco quando è collegato solo un pad; F12 è un tasto di tastiera
- **Il monitor**: immagine da fare: shots/boot-os-monitor.png
- **Schermate**: verificare Select tenuto 3 s sul PowKiddy RGB30; non ancora provato sulla console
- **Crash e log**: cosa significa ogni numero di lampeggi del LED sul Pi Zero W; sono scritti solo i codici dell'RGB30
- **Crash e log**: immagine da fare: shots/boot-os-crash.png
- **Crash e log**: un file di log sulla scheda SD per il Pi Zero W